

neospice: An embeddable C++20 circuit simulator with Python bindings and differential SPICE validation

Subhagato Dutta ¹

¹ Carnegie Mellon University

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Electronic circuit simulation predicts how interconnected components respond to electrical inputs. Researchers use these predictions to examine operating conditions, frequency response, switching behavior and noise before building or measuring a circuit. Integrating a simulator into a larger research program also requires access to circuit construction, simulation settings and results.

neospice is an open-source circuit simulator implemented as a C++20 library, with a command-line interface and Python bindings. It accepts SPICE netlists and provides programmatic circuit construction and structured analysis results. The project combines established circuit-simulation methods and adapted device models with interfaces intended for embedding and experimentation. Its contribution is the software implementation, extension architecture and reproducible comparison infrastructure; it does not claim to introduce the underlying SPICE algorithms or compact-device models.

Statement of need

The intended audience is researchers who need to work with the simulation engine itself while developing automated circuit-characterization programs or experimental device and analysis implementations. Such work can require coordinating circuit state, changing components and inspecting numerical results inside a host application. neospice exposes these operations through C++ objects and Python bindings, with the engine, parser and device adapters maintained in the same repository.

This scope carries a substantial correctness obligation. An API that is easy to embed is useful only when the selected models and analyses behave as documented. neospice therefore treats agreement with a declared ngspice version as an empirical requirement. Unsupported analyses must report an error, and unsuccessful simulations must remain visible to the calling program. A returned array or successful process exit alone is insufficient evidence that a requested simulation completed correctly.

Pending author evidence: the final paper must identify an actual research workflow, its scientific question and its use of these interfaces. The current examples and developer validation experiments are demonstrations; they do not establish external adoption or prior research use.

State of the field

ngspice is an established open-source SPICE simulator with extensive device and analysis support ([ngspice project](#), [n.d.-b](#)). It also provides a shared-library interface, so embedding is not unique to neospice ([ngspice project](#), [n.d.-a](#)). PySpice provides Python interfaces to ngspice

39 and Xyce and integrates simulation with Python data analysis (PySpice project, n.d.). Xyce is
40 an independent, modular C++ circuit simulator developed at Sandia National Laboratories,
41 including large and parallel simulations (Sandia National Laboratories, n.d.). Sandia also
42 documents a C interface and Python wrappers (Sholander & Schiek, 2020). These projects
43 provide existing routes for researchers who primarily need to run circuits or automate mature
44 simulators.

45 The proposed justification for a separate implementation is direct access to a C++ circuit
46 and device architecture whose internal state and extension points can be changed together
47 with the host research program. That choice exchanges the maturity of an existing engine
48 for control over its implementation and interfaces. It does not establish that wrapping or
49 contributing to ngspice is inadequate for ordinary simulation tasks. The final research workflow
50 must demonstrate why this trade-off was necessary in the author's application.

51 Software design

52 The parser and programmatic interface construct circuits used by shared analysis drivers.
53 Device adapters expose operations for setup, equation evaluation, small-signal stamping and
54 supported noise contributions. Common state and integration interfaces allow the drivers
55 to coordinate device history during transient simulation. Descriptor-based migration tools
56 assist with adapting ngspice-derived model code; translation and code generation do not
57 independently validate a model's behavior.

58 The sparse solver layer separates equation construction from numerical solution. The current
59 automatic real-solver policy uses a separate minimum-degree-ordered LU implementation for
60 sufficiently large linear circuits and retains the Sparse-derived path for other cases. Complex
61 operations use the latter path. Different elimination orders can produce different floating-point
62 results, even when a mathematical solution is unique. Solver selection is consequently a
63 compatibility decision as well as a performance decision. No new ordering algorithm or general
64 speed advantage is claimed here.

65 AC analysis caches frequency-independent conductance and capacitance stamps, then assembles
66 the frequency-dependent system and applies device-specific corrections at each frequency. Noise
67 analysis uses separate gain and adjoint systems. These are implementation choices with costs
68 as well as benefits: ngspice can reuse existing factors for a transpose solve, whereas neospice's
69 current noise path factors separate systems. Paired, validated measurements are needed before
70 attributing a performance benefit to either arrangement.

71 Validation combines analytic checks, API and tooling tests, and differential comparisons against
72 the checksum-pinned ngspice 47 release. Comparisons reject empty, nonfinite, unsuccessful
73 and structurally invalid results, require the intended signals and retain error diagnostics.
74 Adaptive transient outputs are compared on both time grids with a documented interpolation
75 rule. Primary vendor-model fixtures and separately generated isolated/driven variants retain
76 independent outcomes, including failures. Fixture construction is frozen before execution, and
77 the experiment records source, binary, dependency and runtime hashes.

78 Research impact statement

79 The present evidence demonstrates a substantial implementation and a reusable validation
80 process, but does not yet establish a submission-ready research result. The frozen operating-
81 point experiment contains 34,908 primary cases and 32,451 separately planned variants. These
82 are generated cases rather than independently certified model definitions: duplicate names,
83 lexical scope and isolation dependencies still require interpretation. Minimal operating-point
84 fixtures also cannot establish transient, AC or noise correctness.

85 The working candidate targets ngspice 47 exclusively, including its MES multiplier, AM-source

and frequency-grid semantics. An operating-point regression failure and corpus mismatches remain. VDMOS AC and noise are explicitly unsupported. These boundaries must be resolved or justified within a declared support scope before final claims are made.

Pending final evidence: replace this draft status account with the actual research workflow, accuracy-qualified benchmark results, final compatibility accounting and independently checked installation instructions. Preserve any remaining limitations. No publication, outside user, performance improvement or completed human review is asserted by this draft.

AI usage disclosure

The author reports using Codex and Claude Code during development. During the documented JOSS preparation, Codex inspected implementation and reference source, edited numerical and validation code, added tests, ran comparisons and prepared documentation and this manuscript draft. Automated verification includes regression tests, reference comparisons and sanitizer probes, with failures retained in the repository evidence. These checks do not substitute for human review or establish correctness of all generated content.

Pending author confirmation: record the tools, models and versions used throughout development, the historical scope of assistance, the human author's core design decisions and review responsibilities, and approval of the final software and manuscript. Claude Code's specific contributions have not yet been supplied.

Acknowledgements

The implementation builds on work by the SPICE3, ngspice, Sparse and SuiteSparse communities and the original device-model authors. Distribution notices and model-specific attribution are maintained in the repository.

Pending author confirmation: funding, sponsor involvement, conflicts of interest, additional authors and individual contributions. An absence of these has not been assumed.

References

- ngspice project. (n.d.-a). *Ngspice as a Shared Library*. Retrieved September 11, 2026, from <https://ngspice.sourceforge.io/shared.html>
- ngspice project. (n.d.-b). *Ngspice Documentation*. Retrieved September 10, 2026, from <https://ngspice.sourceforge.io/docs.html>
- PySpice project. (n.d.). *PySpice 1.5: Overview*. Retrieved September 10, 2026, from <https://pyspice.fabrice-salvaire.fr/releases/v1.5/overview.html>
- Sandia National Laboratories. (n.d.). *About Xyce*. Retrieved September 10, 2026, from <https://xyce.sandia.gov/about-xyce/>
- Sholander, P. E., & Schiek, R. L. (2020). *Application Note: Mixed Signal Simulation with Xyce 7.1* (No. SAND2020-5614). Sandia National Laboratories. <https://xyce.sandia.gov/files/xyce/AppNote-MixedSignal.pdf>